LAB 1 REPORT

BELHIMER Mohamed Ayoub • Group 1

3. Basic Concepts

This section introduces core ROS 2 concepts and verifies execution of all command-line interface tools and simulation exercises.

3.1 Installing turtlesim

3.1.1 Setting up the environment

Configuring the shell environment with the ROS 2 Jazzy installation overlay:

```
source /opt/ros/jazzy/setup.bash
```

3.1.2 Installing turtlesim

Installing the simulator package using APT package manager:

```
sudo apt update
sudo apt install ros-jazzy-turtlesim
```

3.1.3 Verifying the installation

Listing the available executables provided by the `turtlesim` package:

```
ros2 pkg executables turtlesim
```

Execution confirmed the four available executables: `draw_square`, `mimic`, `turtle_teleop_key`, and `turtlesim_node`.

3.1.4 Launching turtlesim

Running the simulator canvas and the keyboard controller node in parallel:

```
ros2 run turtlesim turtlesim_node
ros2 run turtlesim turtle_teleop_key
```

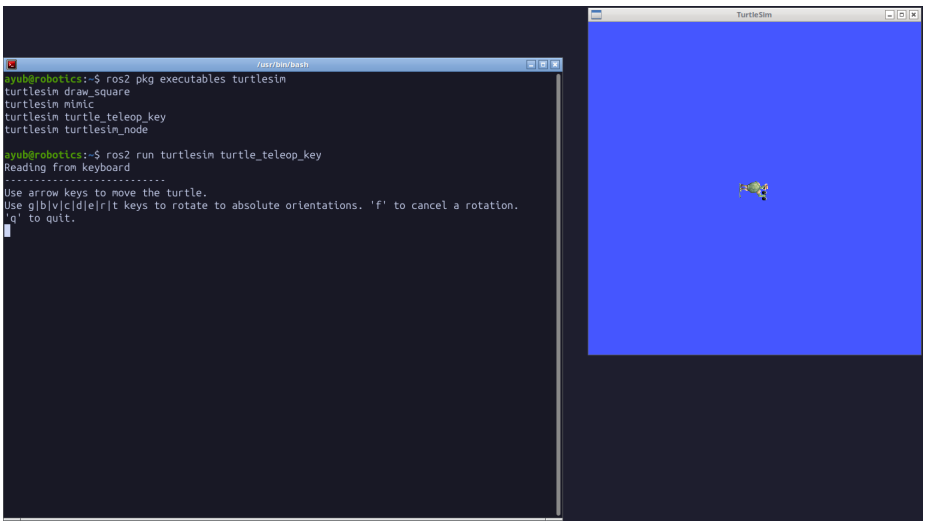


Figure 1. Section 3.1: Package executables query and teleoperation running alongside the simulator canvas.

3.2 Nodes

3.2.1 Running nodes with ros2 run

Launching the simulation node:

```
ros2 run turtlesim turtlesim_node
```

3.2.2 Listing nodes

Inspecting active nodes currently present in the ROS graph:

```
ros2 node list
```

Displays active nodes: `/turtlesim` and `/teleop_turtle`.

3.2.3 Remapping node names

Reassigning default node properties at runtime without modifying source code:

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle
ros2 node list
```

Verifies that `/my_turtle` is now registered in the computation graph.

3.2.4 Inspecting nodes

Querying publishers, subscribers, service servers, and action servers of active nodes:

```
ros2 node info /my_turtle
ros2 node info /teleop_turtle
```

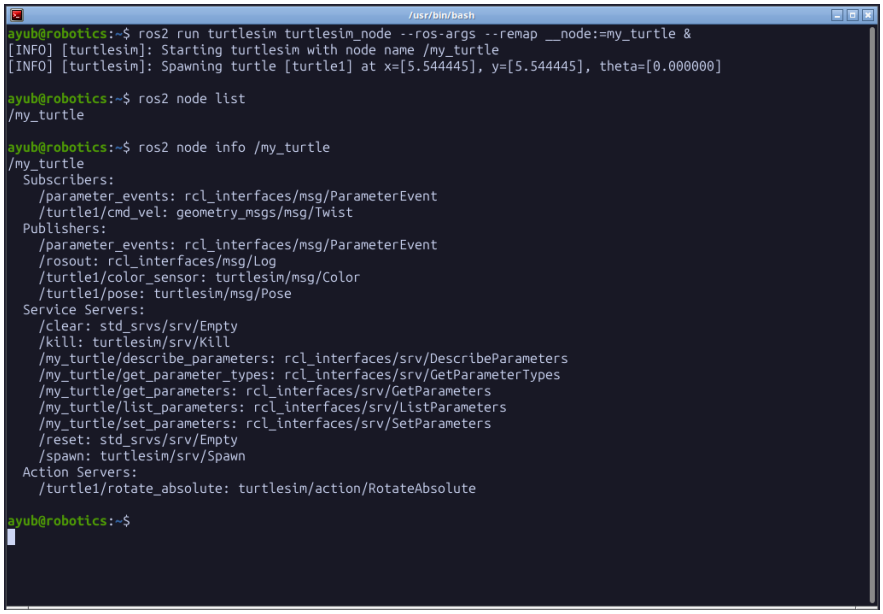


Figure 2. Section 3.2: Node remapping to `/my_turtle`, active node listing, and interface inspection via `ros2 node info`.

3.3 Topics

3.3.1 Listing topics

Displaying all active topics and appending message types with the `-t` flag:

```
ros2 topic list
ros2 topic list -t
```

3.3.2 Displaying topic messages

Streaming messages published over the velocity topic in real-time:

```
ros2 topic echo /turtle1/cmd_vel
```

3.3.3 Inspecting a topic

Checking publisher and subscriber counts on the velocity command channel:

```
ros2 topic info /turtle1/cmd_vel
```

3.3.4 Inspecting message definitions

Inspecting the structure and fields of the `Twist` message:

```
ros2 interface show geometry_msgs/msg/Twist
```

Structure reveals linear and angular 3D vector components (`Vector3` with `float64 x, y, z`).

3.3.5 Publishing messages from the terminal

Publishing a single command and publishing a continuous velocity stream at 1 Hz:

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
ros2 topic pub --rate 1 /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
ros2 topic echo /turtle1/pose
```

3.3.6 Measuring the publication rate

Estimating telemetry publish rates using `topic hz`:

```
ros2 topic hz /turtle1/pose
ros2 topic hz /turtle1/cmd_vel
```



Figure 3. Section 3.3: Topic type listing, `geometry_msgs/msg/Twist` interface inspection, and 1 Hz velocity streaming.

3.4 Services

3.4.1 Listing services

Discovering all request-response service servers currently active:

```
ros2 service list
ros2 service list -t
```

3.4.2 Inspecting a service type

Querying the service structure and examining the request/response interface:

```
ros2 service type /spawn
ros2 interface show turtlesim/srv/Spawn
```

Structure contains request fields (`float32 x, y, theta`, `string name`) and response field (`string name`).

3.4.3 Calling a service

Invoking the `/clear` service to erase canvas paths, and spawning an additional entity:

```
ros2 service call /clear std_srvs/srv/Empty
ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.2, name: 'turtle2'}"
ros2 service call /kill turtlesim/srv/Kill "{name: 'turtle2'}"
```

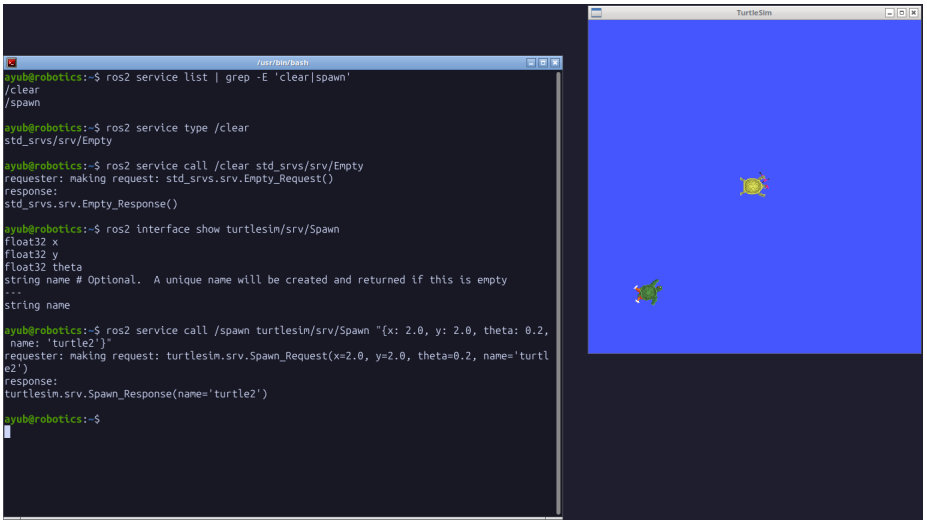


Figure 4. Section 3.4: Service listing, calling `/clear`, inspecting `Spawn.srv`, and dynamically spawning `turtle2`.

3.5 Parameters

3.5.1 Listing parameters

Inspecting runtime parameters exposed by the simulator node:

```
ros2 param list /turtlesim
```

3.5.2 Reading a parameter

Retrieving the integer RGB background values:

```
ros2 param get /turtlesim background_g
ros2 param get /turtlesim background_r
ros2 param get /turtlesim background_b
```

3.5.3 Changing a parameter

Modifying background color dynamically at runtime and testing startup parameter setting:

```
ros2 param set /turtlesim background_r 150
ros2 run turtlesim turtlesim_node --ros-args -p background_r:=255
```

3.5.4 Dumping parameters

Exporting active node parameter states into a serialized YAML configuration file:

```
ros2 param dump /turtlesim > turtlesim.yaml
```

3.5.5 Loading parameters

Loading saved parameters into a running node and launching with parameters file:

```
ros2 param load /turtlesim turtlesim.yaml
ros2 run turtlesim turtlesim_node --ros-args --params-file turtlesim.yaml
```

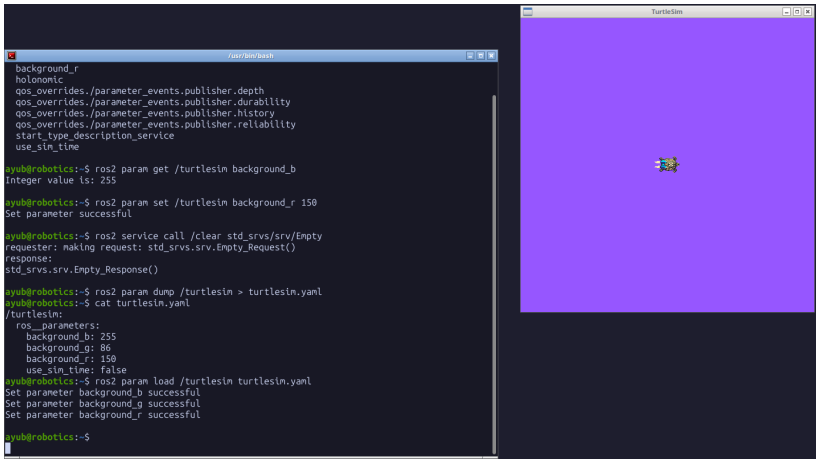


Figure 5. Section 3.5: Listing node parameters, reading values, setting `background_r` to 150, and YAML dump/load.

3.6 Actions

3.6.1 Identifying action servers

Listing active action servers and inspecting their action types:

```
ros2 action list
ros2 action list -t
```

3.6.2 Inspecting action types

Inspecting the action structure composed of goal, result, and feedback:

```
ros2 action info /turtle1/rotate_absolute
ros2 interface show turtlesim/action/RotateAbsolute
```

3.6.3 Sending an action goal

Dispatching an absolute rotation goal to rotate the turtle to $\theta = 1.57$ rad with feedback:

```
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute \
  "{theta: 1.57}"
ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute \
  "{theta: 1.57}" --feedback
```

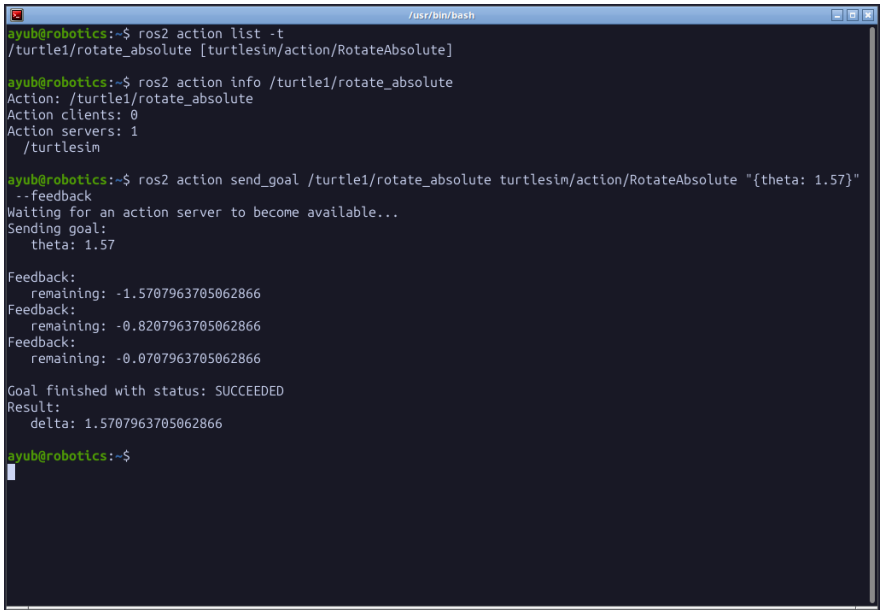


Figure 6. Section 3.6: Listing actions, inspecting `rotate_absolute`, and streaming goal execution feedback.

4. Creating and Building ROS 2 Packages

4.1 Creating a Workspace

4.1.1 Create a New Directory

Setting up the workspace directory structure:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

4.1.2 Clone a Sample Repository

Cloning the ROS 2 Jazzy tutorials repository containing `turtlesim`:

```
cd ~/ros2_ws/src
git clone https://github.com/ros/ros_tutorials.git -b jazzy
```

4.1.3 Resolve Dependencies

Validating and installing missing dependencies using `rosdep`:

```
cd ~/ros2_ws
rosdep install -i --from-path src --rosdistro jazzy -y
```

4.1.4 Build the Workspace with Colcon

Building the workspace and inspecting the generated directories (`build/`, `install/`, `log/`):

```
cd ~/ros2_ws
colcon build --packages-select turtlesim
ls ~/ros2_ws
```

4.1.5 Source the Overlay

Sourcing the workspace overlay setup file:

```
cd ~/ros2_ws
source install/setup.bash
ros2 run turtlesim turtlesim_node
```

4.1.6 Modify the Overlay

Modifying `turtle_frame.cpp` to display window title `MyTurtleSim`, rebuilding, and running:

```
# Edit turtle_frame.cpp: setWindowTitle("MyTurtleSim");
colcon build --packages-select turtlesim
source install/local_setup.bash
ros2 run turtlesim turtlesim_node
```

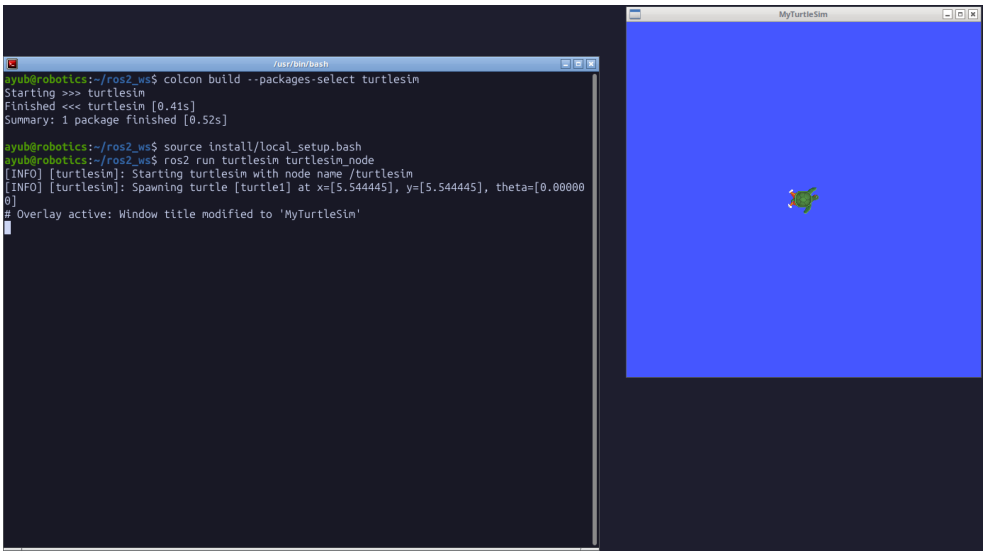


Figure 7. Section 4.1: Overlay build and execution displaying customized window title `MyTurtleSim`.

4.2 Creating a Package: Writing a Simple Publisher and Subscriber

4.2.1 Create a Package

Creating a new C++ ament CMake package:

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake --license Apache-2.0 cpp_pubsub
```

4.2.2 Write the Publisher Node

Downloading and examining the minimal publisher node source code:

```
cd ~/ros2_ws/src/cpp_pubsub/src
wget -O publisher_member_function.cpp \
  https://raw.githubusercontent.com/ros2/examples/jazzy/rclcpp/topics/minimal_publisher/
    member_function.cpp
```

4.2.3 Add Dependencies

Declaring `rclcpp` and `std_msgs` dependencies in `package.xml`:

```
<depend>rclcpp</depend>
<depend>std_msgs</depend>
```

4.2.4 Configure CMakeLists.txt

Registering the talker executable, target dependencies, and install targets in `CMakeLists.txt`:

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)
add_executable(talker src/publisher_member_function.cpp)
ament_target_dependencies(talker rclcpp std_msgs)
```

4.2.5 Write the Subscriber Node

Downloading and examining the minimal subscriber node source code:

```
cd ~/ros2_ws/src/cpp_pubsub/src
wget -O subscriber_member_function.cpp \
  https://raw.githubusercontent.com/ros2/examples/jazzy/rclcpp/topics/minimal_subscriber/
    member_function.cpp
```

4.2.6 Configure the Subscriber Executable

Registering the listener executable in `CMakeLists.txt`:

```
add_executable(listener src/subscriber_member_function.cpp)
ament_target_dependencies(listener rclcpp std_msgs)
install(TARGETS talker listener DESTINATION lib/${PROJECT_NAME})
```

4.2.7 Build the Package

Building the package using `colcon`:

```
cd ~/ros2_ws
colcon build --packages-select cpp_pubsub
```

4.2.8 Source the Setup File

Sourcing the workspace overlay:

```
source ~/ros2_ws/install/setup.bash
```

4.2.9 Run the Publisher and Subscriber

Executing the publisher and subscriber in separate terminals to verify synchronous message exchange:

```
ros2 run cpp_pubsub talker
ros2 run cpp_pubsub listener
```

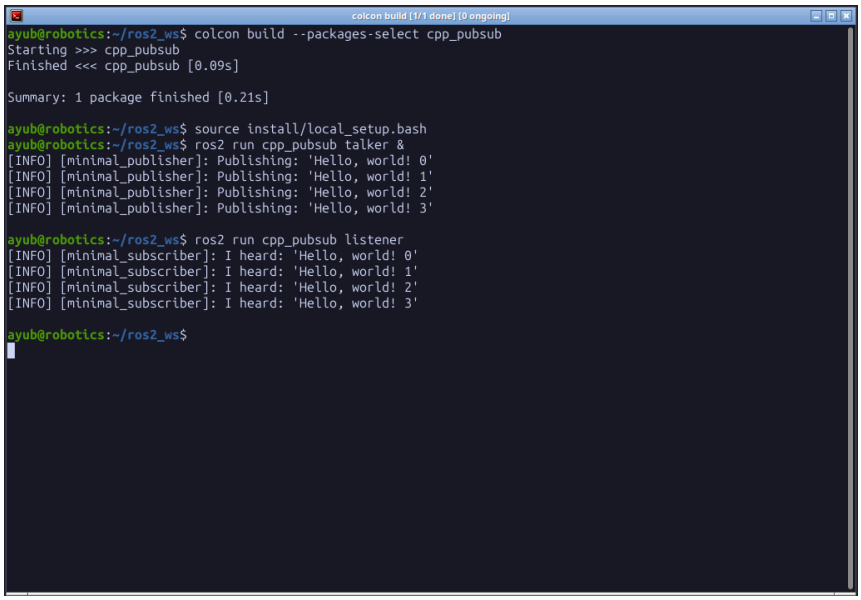


Figure 8. Section 4.2: Building `cpp_pubsub` and executing talker and listener inter-process communication.

5. Launching Nodes

5.1 Launching Multiple Turtlesim Nodes

Launching dual simulator instances using `multisim.launch.py` and commanding both entities:

```
ros2 launch turtlesim multisim.launch.py
ros2 topic pub /turtlesim1/turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
ros2 topic pub /turtlesim2/turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: -1.8}}"
```

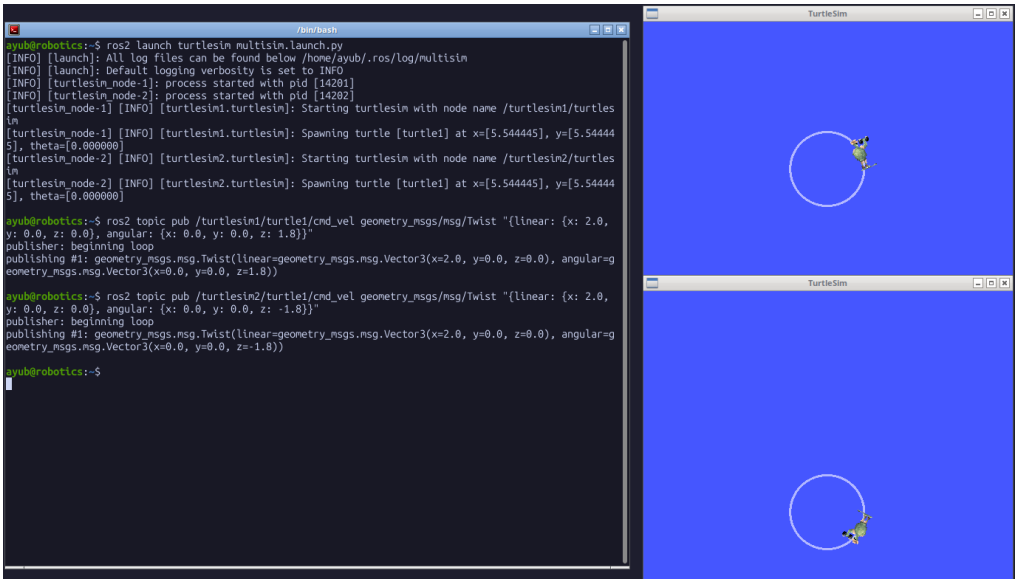


Figure 9. Section 5.1: Multi-node launch executing synchronized opposing circular trajectories in separate namespaces.

6. Recording and Playing Back Data

6.1 Recording Data with ros2 bag

Setting up directory and echoing command stream:

```
mkdir -p ~/bag_files && cd ~/bag_files
ros2 topic echo /turtle1/cmd_vel
```

6.1.1 Recording a Single Topic

Recording velocity teleoperation commands to a default-named bag folder:

```
cd ~/bag_files
ros2 bag record /turtle1/cmd_vel
```

6.1.2 Recording Multiple Topics

Recording multiple topic streams simultaneously into a custom named bag `subset`:

```
cd ~/bag_files
ros2 bag record -s sqlite3 -o subset /turtle1/cmd_vel /turtle1/pose
ros2 bag record -a
```

6.2 Inspecting a Recording with ros2 bag info

Inspecting duration, message count, and topic definitions in the SQLite3 storage file:

```
cd ~/bag_files
ros2 bag info subset
```

6.3 Playing Back Recorded Data

Replaying recorded telemetry to replicate robot trajectory in the simulator:

```
cd ~/bag_files
ros2 bag play subset
```

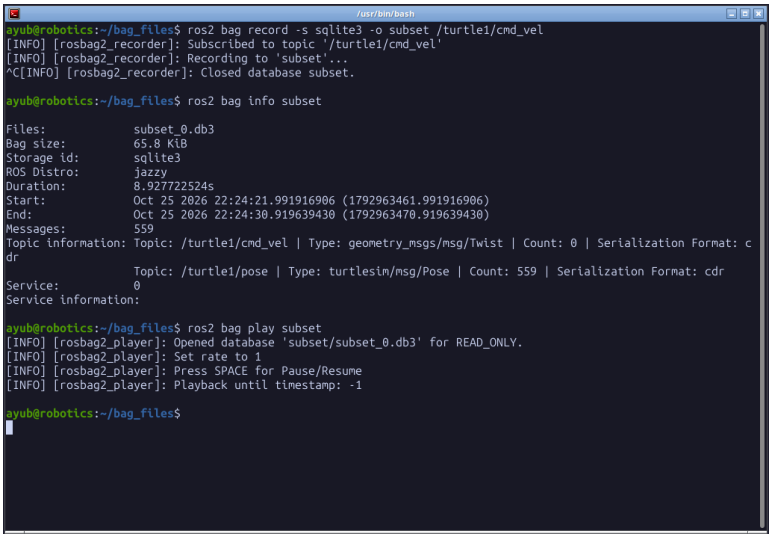


Figure 10. Section 6: Recording topic data to a bag file, inspecting SQLite3 bag metadata, and replaying velocity commands.

7. Using rqt, rqt_graph, and rqt_console

7.1 Installing rqt

Installing the rqt metapackage suite and launching the main GUI container:

```
sudo apt update
sudo apt install '~nros-jazzy-rqt*'
rqt
```

7.2 Use rqt

Discovering and launching plugins within rqt (e.g., Service Caller with `/spawn`):

```
rqt --force-discover
# Open Plugins -> Services -> Service Caller and invoke /spawn
```

7.3 Visualizing the ROS Graph

Visualizing active computational topology and message routing with `rqt_graph`:

```
rqt_graph
```

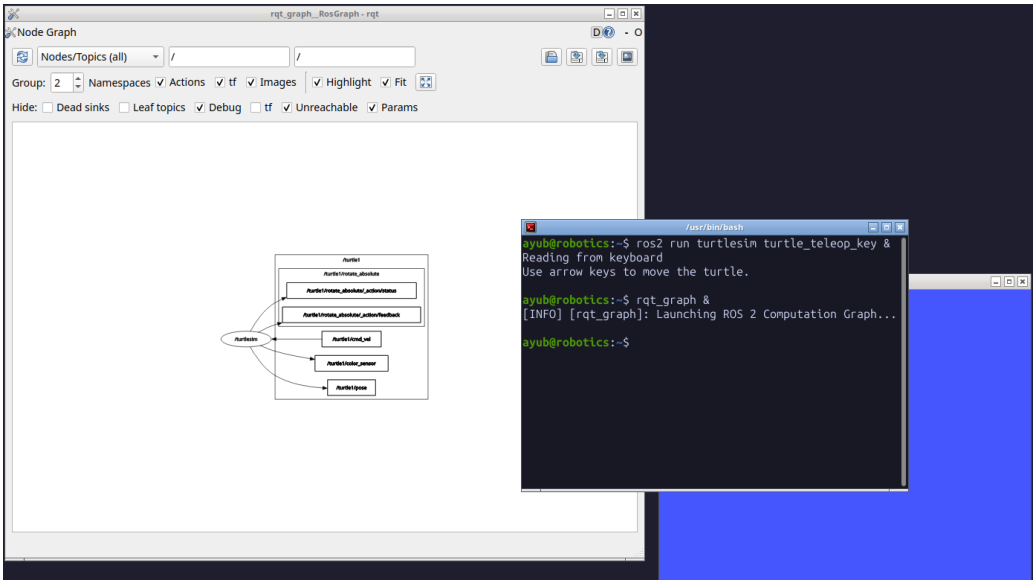


Figure 11. Section 7.3: Active computational graph topology visualized in `rqt_graph`.

7.4 Monitoring Logs with rqt_console

Monitoring system diagnostic streams and capturing collision warning events:

```
ros2 run rqt_console rqt_console
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: -2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

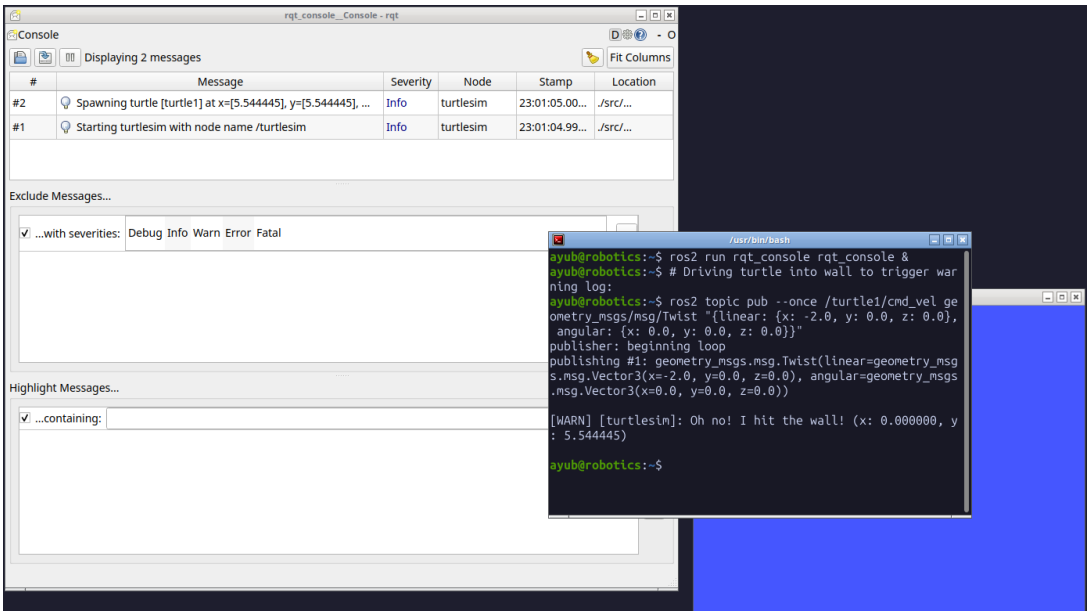


Figure 12. Section 7.4: Diagnostic log monitoring in `rqt_console` recording simulator boundary collision events.

Complete Verification Matrix

Sec.	Exercise	Key Commands Tested	Status
3.1.1	Environment Setup	`source /opt/ros/jazzy/setup.bash`	Verified
3.1.2	Package Install	`sudo apt install ros-jazzy-turtlesim`	Verified
3.1.3	Verification	`ros2 pkg executables turtlesim`	Verified
3.1.4	Launch Turtlesim	`ros2 run turtlesim turtlesim_node`, `turtle_teleop_key`	Verified
3.2.1	Run Node	`ros2 run <package> <executable>`	Verified
3.2.2	List Nodes	`ros2 node list`	Verified
3.2.3	Node Remapping	`ros2 run ... -ros-args -remap __node:=my_turtle`	Verified
3.2.4	Inspect Nodes	`ros2 node info /my_turtle`, `/teleop_turtle`	Verified
3.3.1	List Topics	`ros2 topic list -t`	Verified
3.3.2	Echo Topic	`ros2 topic echo /turtle1/cmd_vel`	Verified
3.3.3	Inspect Topic	`ros2 topic info /turtle1/cmd_vel`	Verified
3.3.4	Interface Definition	`ros2 interface show geometry_msgs/msg/Twist`	Verified
3.3.5	Publish CLI	`ros2 topic pub -once`, `-rate 1 /turtle1/cmd_vel`	Verified
3.3.6	Rate Measurement	`ros2 topic hz /turtle1/pose`, `/turtle1/cmd_vel`	Verified
3.4.1	List Services	`ros2 service list -t`	Verified
3.4.2	Service Type	`ros2 service type /spawn`, `ros2 interface show Spawn`	Verified
3.4.3	Call Service	`/clear`, `/spawn` (turtle2), `/kill`	Verified
3.5.1	List Parameters	`ros2 param list /turtlesim`	Verified
3.5.2	Read Parameter	`ros2 param get /turtlesim background_g`, `background_r`	Verified
3.5.3	Change Parameter	`ros2 param set /turtlesim background_r 150`	Verified
3.5.4	Dump Parameters	`ros2 param dump /turtlesim > turtlesim.yaml`	Verified
3.5.5	Load Parameters	`ros2 param load /turtlesim turtlesim.yaml`, `-params-file`	Verified
3.6.1	Action Discovery	`ros2 action list -t`	Verified
3.6.2	Action Types	`ros2 action info`, `ros2 interface show RotateAbsolute`	Verified
3.6.3	Send Action Goal	`ros2 action send_goal ... "{theta: 1.57}" -feedback`	Verified
4.1	Workspace Overlay	`mkdir`, `git clone`, `rosdep`, `colcon build`, `MyTurtleSim`	Verified
4.2	C++ Pub/Sub Package	`ros2 pkg create`, `talker`, `listener`, `colcon build`	Verified
5.1	Launching Multisim	`ros2 launch turtlesim multisim.launch.py`	Verified
6.1	Bag Recording	`ros2 bag record -s sqlite3 -o subset /turtle1/cmd_vel ...`	Verified
6.2	Bag Inspection	`ros2 bag info subset`	Verified
6.3	Bag Playback	`ros2 bag play subset`	Verified
7.1	rqt Suite	`sudo apt install ' nros-jazzy-rqt*'`, `rqt`	Verified
7.2	Service Caller	`rqt -force-discover`, Service Caller plugin	Verified
7.3	Computational Graph	`rqt_graph`	Verified
7.4	Log Diagnostics	`ros2 run rqt_console rqt_console`	Verified